Introduction: Systemic Failures in Agentic Architectures

Why LLM Agents Fail, and What Control Theory Teaches Us About Fixing Them

Melvyn J. White

https://www.linkedin.com/in/melvyn-white-44450011/

> *"The machine is a tool - no more, no less. It does what man tells it to do... But it is a tool of a peculiar kind: once set in motion, it proceeds on its own, following its own logic, indifferent to human desires or even human survival."*
>
> - David S. Landes, *The Unbound Prometheus* (1969)

The transition from LLMs as chatbots to LLMs as autonomous agents represents a fundamental shift in failure modes. A chatbot that hallucinates produces a wrong answer; an agent that hallucinates may execute a wrong action, triggering real-world consequences that cannot be undone.

This introduction examines the **systemic issues** inherent in agentic architectures - not bugs in specific implementations, but structural problems that emerge from the fundamental design of LLM-based agents [1][3]. We apply **control theory** - the mathematical framework that made industrial automation reliable - to understand why agents fail and how to fix them.

1. The Agent Paradigm Shift

Traditional LLM usage is **stateless** and **passive**: user provides input, model provides output, interaction ends. Agentic LLM usage is **stateful** and **active**: the model maintains context across steps, makes decisions that trigger real actions, observes results, and iterates [1].

Dimension	Chatbot (Passive)	Agent (Active)
State	Stateless (each call independent)	Stateful (maintains context)
Actions	Text generation only	Tool use, code execution, API calls
Consequences	Wrong text (correctable)	Wrong actions (may be irreversible)
Termination	Always terminates	May loop indefinitely
Failure mode	Hallucinated content	Hallucinated actions with real effects

Table 1: Chatbot vs. Agent Paradigms

2. Systemic Issues in Agentic Architectures

2.1 Unbounded Execution: The Stability Problem

The most fundamental issue: **agents can fail to terminate**. Unlike a function call that always returns, an agent loop may continue indefinitely - retrying failed actions, exploring unproductive paths, or cycling between states without progress [4].

The Core Question: Self-Correcting or Self-Reinforcing?

Every system has a fundamental character: when disturbed, does it **return to balance**, or does it **spiral out of control**? Consider two everyday examples:

THERMOSTAT Self-Correcting	MICROPHONE FEEDBACK Self-Reinforcing
Room too hot -> heater turns off	Sound enters microphone
Room too cold -> heater turns on	Speaker amplifies -> mic picks up more
Deviation triggers correction	**Deviation triggers amplification**
System returns to target	System produces screech

The thermostat has **negative feedback** *(deviations opposed). The microphone has* **positive feedback** *(deviations amplified).*

Definition: BIBO Stability (Bounded-Input-Bounded-Output)

A system is **BIBO stable** if: *for every bounded input, the output remains bounded*.

For agents: A reasonable request produces a response in finite time with finite cost - **not** infinite loops or runaway spending.

What Determines Stability? The Role of "Poles"

Control engineers use **poles** to quantify a system's tendency toward balance or runaway. Think of poles as measuring the **dominant forces** acting on the system:

Type of Force	What Happens	Technical Term	Result
Restoring force	Pushes back toward equilibrium	Pole in Left Half-Plane (LHP)	**STABLE**
Runaway force	Pushes further from equilibrium	Pole in Right Half-Plane (RHP)	**UNSTABLE**

Table 2: Poles as Forces

The thermostat has only **restoring forces** - all poles in LHP. The microphone has a **runaway force** - a pole in RHP.

The Critical Insight

A system is stable if **ALL** poles are in LHP. **Even ONE** pole in RHP makes the entire system unstable. One unchecked amplifying process will eventually dominate.

How This Applies to Agents

Each design choice creates either a **restoring force** or a **runaway force**:

Agent Design Choice	Type of Force	Consequence
+ Iteration limit enforced	Restoring (LHP)	Must terminate
+ Cost limit enforced	Restoring (LHP)	Spending bounded
+ External verification	Restoring (negative feedback)	Errors corrected
- No iteration limit	Runaway (RHP)	May loop forever
- No cost limit	Runaway (RHP)	Costs explode
- Errors unchecked	Runaway (positive feedback)	Errors compound

Table 3: Agent Design Choices as Forces

Figure 1: Visualizing Stability - The Complex Plane

Engineers visualize pole locations on the **complex s-plane**. The vertical line divides the plane: left = stable, right = unstable.

LEFT HALF-PLANE Restoring Forces Dominate	RIGHT HALF-PLANE Runaway Forces Dominate
Disturbances **decay** over time	Disturbances **grow** over time
System **returns to equilibrium**	System **spirals out of control**
+ Agent converges to answer	- Agent loops endlessly
+ Costs stay bounded	- Costs grow without bound
+ Always terminates	- May never terminate

The thick line is the stability boundary. All poles must be on the left.

Stability Checklist

To guarantee stability, every potential runaway force must have a limit:

- ☐ **Max iterations** - bounds the retry loop
- ☐ **Max tokens** - bounds context growth
- ☐ **Max cost** - bounds spending
- ☐ **Max time** - bounds duration
- ☐ **External verification** - corrects errors

☐ **Exponential backoff** - dampens retries

Warning: A missing limit is a runaway force. **One unchecked runaway force makes the entire system unstable.**

2.2 Error Propagation: Positive Feedback

In multi-step reasoning, a hallucination in step 1 becomes a premise for step 2. Errors **compound** - like microphone screech, small errors become large failures [5][6].

2.3-2.6 Other Systemic Issues

Issue	Control Analogy	Consequence
Unobservability	Hidden state	Cannot diagnose or correct [15]
Poor calibration	Bad sensor	False confidence in wrong answers [5]
No verification	Open-loop control	No error correction [8][9]
Multi-agent echo	Coupled positive feedback	Mutual error amplification [10][11]

Table 4: Other Systemic Issues

3. Does AI Self-Reflection Count as Feedback?

Self-reflection uses **the same model** that generated the output - like checking your own homework with the same faulty reasoning. The model that hallucinated will evaluate the hallucination as correct [7].

Verdict: Self-reflection is **weak negative feedback** - catches obvious errors but cannot reliably detect hallucinations.

Feedback Type	Source	Strength
Self-reflection	Same LLM	**Weak**
Cross-model	Different LLM	Medium
External verification	Non-LLM (compiler, tests)	**Strong**
Human review	Human expert	**Strong**

Table 5: Feedback Mechanisms

4. The Control Theory Solution

Problem	Solution	Implementation

Runaway forces	Add restoring forces	Hard limits on iterations, tokens, cost, time
Positive feedback	Add negative feedback	External verification at each step
Open-loop	Close the loop	Verification oracles, formal methods [9]
Echo chamber	Break loop, add damping	Supervisor pattern, independent checks

Table 6: Control Theory Solutions

5. Conclusion

Control theory provides the principled framework: **identify every runaway force and add a corresponding restoring force**.

Design Principle

Stability is the minimum requirement. **Eliminate all runaway forces. Close the loop with verification.**

References

[1] Yao et al. (2023). ReAct. ICLR. arXiv:2210.03629

[2] Shinn et al. (2023). Reflexion. NeurIPS. arXiv:2303.11366

[3] Wang et al. (2024). Survey on LLM Agents. Frontiers of CS.

[4] Anthropic (2024). Building Effective Agents.

[5] Kadavath et al. (2022). Language Models Know What They Know. arXiv:2207.05221

[6] Ganguli et al. (2022). Red Teaming Language Models. arXiv:2209.07858

[7] Zheng et al. (2023). LLM-as-a-Judge. NeurIPS. arXiv:2306.05685

[8] Chen et al. (2021). Evaluating LLMs on Code. arXiv:2107.03374

[9] Wolfram (2023). ChatGPT Gets Wolfram Superpowers.

[10] Wu et al. (2023). AutoGen. arXiv:2308.08155

[11] Du et al. (2023). Multiagent Debate. arXiv:2305.14325

[14] Shinners (1998). Modern Control System Theory. Wiley.

[15] Nyquist (1932). Regeneration Theory. Bell Sys. Tech. J.

--- End of Introduction ---